

Data Wrangling 🤠

String Manipulation Functions

Lesson 16

Where We Left Off

- Last lesson, `csv.DictReader` handed us real data — but **every value came back as a string**
- Real-world text is often messy: extra whitespace, inconsistent casing, stray punctuation
- Today: a toolkit of `str` methods for cleaning and transforming that text

```
>>> row["Title"]  
"  Bohemian Rhapsody  \n"
```

Strings Are Immutable

- `str` methods never change the original string — they **return a new one**
- You must **capture the result** (usually by reassigning) or it's lost

```
>>> title = "bohemian rhapsody"
>>> title.upper()
'BOHEMIAN RHAPSODY'
>>> title
'bohemian rhapsody'
>>> title = title.upper()
>>> title
'BOHEMIAN RHAPSODY'
```

`.strip()` – Removing Whitespace

- Removes leading and trailing whitespace (spaces, tabs, newlines)
- Does **not** touch whitespace in the middle

```
>>> "  Bohemian Rhapsody \n".strip()
'Bohemian Rhapsody'
>>> "  Uptown    Funk  ".strip()
'Uptown    Funk'
```

- `.lstrip()` / `.rstrip()` strip from just one side

`.upper()`, `.lower()`, `.title()`

- Change casing without touching anything else

```
>>> "queen".upper()  
'QUEEN'  
>>> "QUEEN".lower()  
'queen'  
>>> "bohemian rhapsody".title()  
'Bohemian Rhapsody'
```

- Case conversion makes comparisons **reliable**:

```
>>> "Queen" == "queen"  
False  
>>> "Queen".lower() == "queen".lower()
```

`.replace(old, new)`

- Replaces **every** occurrence of `old` with `new` in a new string

```
>>> "Don't Stop Believin'".replace("'", "")  
'Dont Stop Believin'  
>>> "I like rock music. Rock is great.".replace("Rock", "Pop")  
'I like rock music. Pop is great.'
```

- Notice: `.replace()` is case-sensitive — `"rock"` (lowercase) wasn't touched

`.split(sep)` — Revisited

- Breaks a string into a **list** of pieces around a separator
- We used this last lesson to pull a field out of a combined string

```
>>> "Blinding Lights | The Weeknd | 2019".split(" | ")
['Blinding Lights', 'The Weeknd', '2019']
>>> "Uptown Funk".split(" ")
['Uptown', 'Funk']
>>> "  dua  lipa  ".split()
['dua', 'lipa']
```

- Calling `.split()` with **no argument** splits on any whitespace and ignores extra spaces

`.join(list)` — The Reverse of

`.split()`

- Called **on the separator**, glues a list of strings together

```
>>> " | ".join(["Blinding Lights", "The Weeknd", "2019"])
'Blinding Lights | The Weeknd | 2019'
>>> ", ".join(["Uptown Funk", "Levitating", "Blinding Lights"])
'Uptown Funk, Levitating, Blinding Lights'
```

- `sep.join(pieces)` — read it as "join `pieces` together **using** `sep`"

`.startswith()` / `.endswith()`

- Check the beginning or end of a string — return a `bool`

```
>>> "song.mp3".endswith(".mp3")
True
>>> "song.mp3".endswith(".wav")
False
>>> "hip-hop".startswith("hip")
True
```

- Handy for filtering rows without slicing or indexing by hand

Chaining String Methods

- Since every method returns a new string, you can call **another method** directly on the result

```
>>> "  BOHEMIAN RHAPSODY \n".strip().lower().title()  
'Bohemian Rhapsody'
```

- Read left to right: strip, *then* lowercase, *then* title-case the result of that
- Order matters! `.title().strip()` on the same input behaves differently once whitespace is involved

String Method Cheat Sheet

Method	What it does
<code>.strip()</code>	remove leading/trailing whitespace
<code>.upper()</code> / <code>.lower()</code> / <code>.title()</code>	change casing
<code>.replace(old, new)</code>	swap every <code>old</code> for <code>new</code>
<code>.split(sep)</code>	string → list of pieces
<code>sep.join(list)</code>	list of pieces → string
<code>.startswith(x)</code> / <code>.endswith(x)</code>	<code>bool</code> – does it begin/end with <code>x</code> ?

Concept Check

```
title = "  Bohemian Rhapsody \n"  
  
title.strip()  
title.upper()  
title  
title.strip().upper()
```

What is the **value** of each expression?



Activity: Clean Up the Playlist

Working in the REPL (or a scratch file), transform each `raw` value into the matching `target` using **only string methods** – no manual slicing.

```
raw      = "  BLINDING LIGHTS  "  
target   = "Blinding Lights"  
  
raw      = "The Weeknd"  
target   = "the_weeknd"  
  
raw      = "TRK-004821 "  
target   = "TRK004821 "
```

- Chain as many methods as you need on one line

6. ... with `target` ...



Activity: Stretch Challenge

Given a messy row pulled from a playlist CSV, build a clean, human-readable summary string.

```
row = {"Title": "  levitating\n", "Artist": "Dua Lipa", "Plays": "1245000000"}
```

```
# target output:
```

```
"LEVITATING by Dua Lipa – 1,245,000,000 plays"
```

Hint: you'll need `.strip()`, `.upper()`, string concatenation – and `f"{n:,}"` to add commas to a number.

Summary

- Strings are **immutable** — every method returns a **new** string; reassign to keep it
- `.strip()` / `.upper()` / `.lower()` / `.title()` clean up whitespace and casing
- `.replace()` swaps characters or substrings; `.split()` / `.join()` convert between `str` and `list`
- `.startswith()` / `.endswith()` check the ends of a string and return a `bool`
- Methods **chain** — read left to right as a pipeline of transformations